# Arpan Pathak

**Senior Software Engineer** | **AI Infrastructure** | **AI Inference** | **Distributed Systems** | **GPU Cloud**

Seattle, WA · +1 (206) 306-6059 · arpan.pathak47@gmail.com · arpanpathak.github.io
github.com/arpanpathak · linkedin.com/in/arpan-pathak-018879424 · youtube.com/@ArpanPathak

## SUMMARY

Senior Software Engineer with 8+ years of experience building distributed systems, cloud infrastructure, and AI inference platforms at Microsoft, Oracle, Amazon, and Razorpay. Designed and operated production platforms end to end: Kubernetes networking and operators, transactional datastores, payments, and inference serving. Strong in Rust, Go, modern C++, and Python. Currently building and benchmarking GPU computing workloads across edge GPUs and second-hand/refurbished graphics cards, while learning CUDA profiling, inference benchmarking, network and storage performance analysis, and how to write idiomatic CUDA-Rust for tiled parallel algorithms.

## CURRENT PROJECTS

### Thor Thunder Tigress Platform

**Live** · **GitHub** · **Book**

Rust · Python · CUDA · vLLM · SGLang · llama.cpp · TensorRT Edge-LLM · MCP · Tailscale

- Built a **full-stack hardware and software platform** with SDKs and CLIs: a 5-node edge GPU cluster, a data-curation pipeline for QLoRA fine-tuning, a multi-engine inference layer (vLLM, SGLang, Ollama, llama.cpp, TensorRT Edge-LLM), and OpenAI-compatible model routing in async Rust (Tokio) with MCP tool servers, Argon2id/XChaCha20-Poly1305 keyring encryption, and bring-your-own-domain hosting at voltforge.tech.
- Serves Qwen3.8 Flash Next (125B MoE, NVFP4) at **43–45 tokens/s** with **0.19 s** to first token, Nemotron at **65 tokens/s**, and Qwen3.6-35B-A3B NVFP4 at **78 tokens/s**; a 24 GiB KV cache holds **4 concurrent 1M-token windows**; **≈1.05M** NVMe reads/s and **100 Gb/s** per node.
- Benchmarked models on SWE-bench Verified and Terminal-Bench for long-horizon agentic coding; documented in a companion book.

**Hardware:** 5 × NVIDIA Jetson AGX Thor · Blackwell, 2,560 CUDA cores, 96 Tensor Cores, 128 GB unified memory at 273 GB/s, 4 × 25 GbE QSFP28

### StaleyNET: Text-to-Music Generation with Singing

**GitHub** · **Paper** · **Book** · **Hugging Face**

Rust · CUDA · CUDA-Oxide · Transformers · Audio DSP · Whisper · vLLM · Hugging Face

- Built text-prompted music generation from first principles in Rust with **no pretrained components**: log-mel front end, residual k-means audio tokenizer, a Transformer with **hand-derived backpropagation**, and a neural vocoder, with GPU kernels compiled from Rust to PTX by CUDA-Oxide.
- Trained every model on Jetson AGX Thor and Orin NX, scaling to **111M parameters**; diagnosed overfitting (each song drawn up to **1,634×**, generalization gap **0.975**) and fixed it with balanced sampling, capping draws at **28 per song**.
- Added lyric conditioning by transcribing **32,882 clips** with Whisper large-v3 on vLLM; built a Rust training-observability dashboard; released a paper, a book, a crates.io crate, checkpoints and a 3.1 GB dataset.

## EXPERIENCE

### Microsoft · Software Developer II, M365 Security Platform Engineering

**Redmond, WA · Jun 2025 – Aug 2026**

- Built zero-trust sovereign-cloud networking for AKS clusters in France and Germany, replacing the default CNI and kube-proxy with Cilium eBPF network policies to enforce SecNumCloud data residency; moving from iptables rule chains to eBPF hash maps cut per-node networking memory from **~2 GB to ~100 MB** and service-routing CPU by **~70%**. Mutual TLS secures service traffic.
- Wrote Kubernetes operators in Go with Kubebuilder that generate policy-as-code and reconcile state across sovereign-cloud AKS clusters, and cut compute cost **~25%** by moving stateless and batch workloads to spot node pools with the cluster autoscaler.
- Led on-call and observability for production Kubernetes services: established blameless postmortems; defined SLIs/SLOs; built **Prometheus/Grafana** dashboards and alerting; automated remediation and drove root-cause analysis, IaC, and CI/CD release gates.
- Ported and stabilized parser algorithms for the new Information Protection Service v2 in Rust, eliminating buffer-overflow and null-dereference bugs by design and lowering P99 parse latency from **~38 ms to ~22 ms** and memory per worker from **~420 MB to ~260 MB**.
- Built data loss prevention (DLP) and data lifecycle services in C#/.NET, and hardened legacy C++ with RAII.

### Oracle Cloud Infrastructure · Senior Software Engineer

**Seattle, WA · Oct 2024 – Mar 2025**

- Owned the Terraform provider and the Go and Java SDKs for Autonomous Database, replacing legacy control-plane tooling and reducing manual provisioning and operational overhead.
- Built lock-free data structures for high-concurrency read/write paths and automated key rotation with OCI Vault.

### Amazon · Software Development Engineer II

**Seattle, WA and Hyderabad, India · Mar 2021 – Oct 2024**

- Led a team of 3 building programmatic guaranteed (PG) ad deal delivery at under **50 ms** on a Kafka change-data-capture (CDC) pipeline in Kotlin, cutting deal over-delivery by **~30%** and budget overspend by **25%** with real-time monitoring and automated controls.
- Designed and owned Carbon V3, a NoSQL datastore serving the ad publisher platform's transactional (OLTP) needs: built multi-tenant SDKs with client code generation from AWS Smithy models, and an indexing SDK enforcing integrity (uniqueness) constraints with two-phase commit and serializable isolation, sustaining **3K writes/s**.

- Developed an infrastructure-as-code provisioning DSL that generates TypeScript AWS Cloud Development Kit (CDK) modules for VPCs, VPC peering, and PrivateLink, improving developer experience for multi-tenant ad publisher platforms.
- Owned training and serving of deep-learning models and an XGBoost ensemble for Amazon's ad-publisher platform across Prime Video, Twitch, FreeVee, MiniTV, and Amazon Retail using **PyTorch, TensorFlow, and JAX**, lifting ad CTR **5%** and improving campaign efficiency.
- Scaled low-latency BERT inference for Amazon WorkEvents to **5,000 QPS** with ONNX Runtime and INT8 quantization, automating support ticket categorization and routing and cutting triage SLA from 7 days to 2 hours.
- Built Amazon WorkEvents' data platform for financial reporting and pay computation, including **distributed job scheduling infrastructure** for analytics at **minute-level granularity**; delivered a petabyte-scale data lake and warehouse on AWS Glue and Spark, query DSL, typed Python BI/Ops tooling, and cross-platform Python SDK clients and CLIs (Linux, macOS) for data scientists, with dataset provisioning generated as infrastructure-as-code.

**Razorpay** · Software Development Engineer — Bengaluru, India · Aug 2020 – Feb 2021

- Built the Unified Payments Interface (UPI) gateway for stock brokers in Go with Protobuf/gRPC, covering bank-account TPV (third-party validation), UPI Collect push notifications, and UPI Intent.
- Sustained **3M+ transactions/hour** at peak with **99.99% uptime** and **150 ms P99** on AWS and GCP for customers including Zerodha, Groww, Zomato, and Swiggy, with **Prometheus/Grafana** observability, dashboards, and alerting.

**Mindfire Solutions** · Software Engineer — Bhubaneswar, India · Aug 2018 – Feb 2020

- Built Java/Spring full-stack tools and an AR product-search microservice handling 10K+ daily queries.

## PERSONAL PROJECTS

**Driving-CivicSense** Rust, CUDA, TensorRT, YOLOv8/v11 — GitHub · Paper

**Hardware:** NVIDIA Jetson Orin NX 16GB, NVIDIA Jetson Nano, Orange Pi 3B, Raspberry Pi Zero 2 W camera, LiDAR

- On-device intersection-blockage prevention: YOLOv8/v11 INT8 TensorRT at **~12 ms/frame** detects congested intersections and guides drivers to stop and wait even on green to avoid gridlock; Rust MJPEG/UDP server, 50,000 miles of camera/LiDAR data.

**GPU Kubernetes Cluster** Go, Rust, k3s, Kubebuilder, kube-rs, Docker — GitHub

- GPU job scheduling on a heterogeneous k3s cluster of Jetson boards using the NVIDIA Kubernetes device plugin and Container Toolkit, with Go (controller-runtime) and Rust (kube-rs) controllers reconciling a GpuWorkload CRD.

## BOOKS AND OPEN SOURCE

**CUDA Kernels: GPU and Parallel Programming from First Principles** — Book · GitHub

- CUDA C++ and Rust (CUDA-Oxide) kernels, GPU memory hierarchy and the roofline model, optimized reduction, scan, and matrix multiplication, Nsight Compute profiling, NVLink/NVSwitch and NCCL collectives, and Jetson Orin CPU-vs-GPU benchmarks.

**Data Structures & Algorithms of Systems Programming from First Principles** — Book · GitHub

- Systems engineering in idiomatic Rust with animations: memory allocation, CPU caches, false sharing, paging, atomics-based locks and queues, thread pools, retries, rate limiting, TCP/epoll servers, HTTP/1.1, async executors, and the data structures and algorithms behind them.

**CUDA-Oxide** — GitHub

- Contributed GPU kernels and documentation fixes to NVIDIA's Rust-to-CUDA compiler.

## SKILLS

| | |
|---|---|
| **Languages** | Rust, Go, C, C++, Python, SQL, Node.js, TypeScript, Java, Kotlin, C#, Bash |
| **Systems Programming** | Concurrency, multithreading, lock-free data structures, memory safety, async Rust (Tokio, serde), Linux internals, low-latency and performance engineering, debugging and profiling (gdb, perf), unit and integration testing |
| **Performance and Architecture** | Software performance analysis and optimization, computer architecture, GPU architecture, performance modeling |
| **Distributed Systems** | Microservices, gRPC, Protobuf, REST, Kafka, Spark, CDC, consensus, replication, sharding, two-phase commit, fault tolerance, high availability, scalability, distributed job scheduling |
| **Cloud and Infrastructure** | AWS (EC2, EKS, ECS, Lambda, API Gateway, S3, Glue, EMR, SageMaker, VPC, PrivateLink), AWS CDK, Azure (AKS), OCI, GCP, Kubernetes, Kubernetes operators and CRDs, Kubebuilder, Docker, containers, Helm, Terraform, CI/CD, Git, GitHub Actions, Linux, CLI and SDK development, AWS Smithy code generation |
| **Networking** | TCP/IP, HTTP/2, QUIC, mTLS, eBPF, XDP, Cilium, CNI, load balancing, DNS, WireGuard (Tailscale), 25 GbE |
| **GPU and AI Inference** | Deep learning model architectures, CUDA, CUDA-Oxide, TensorRT, TensorRT-LLM, TensorRT Edge-LLM, vLLM, SGLang, llama.cpp, ONNX Runtime, PyTorch, TensorFlow, JAX, XGBoost, INT8/INT4/NVFP4 quantization, KV cache, MoE, NCCL, NVLink, Nsight Compute, NVIDIA Container Toolkit, NVIDIA Kubernetes device plugin, NVIDIA Jetson |
| **Agentic and LLM Systems** | Agent architectures and frameworks, agent harnesses, LLM agent workflows, tool calling, MCP (Model Context Protocol), model-provider APIs, agentic retrieval and RAG, QLoRA fine-tuning, guardrails and policy engines, cross-language API design, evaluation and benchmarking |
| **Databases** | Amazon DynamoDB, PostgreSQL, MySQL, MongoDB, Cassandra, Redis, Amazon Redshift |
| **Observability** | Prometheus, Grafana, Azure Monitor, AWS CloudWatch, logging, metrics and alerting, SLOs, on-call, root-cause analysis |

## EDUCATION

**B.Tech, Computer Science and Engineering** · RCC Institute of Information Technology, Kolkata — 2014 – 2018